

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 05

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

Nº de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 30/09/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio separé las piezas del Holocrón, use una esquina de modelo para generar las demás y mantuve la base sin las piezas para el

Agregamos la lista de los modelos de la siguiente manera a Visual Studio:

```
//Holocron
Model baseholocron;
Model esquinageneral;
```

Y las importamos con LoadModel();

```
//Holocron
baseholocron = Model();
baseholocron.LoadModel("Models/baseholocron.obj");
esquinageneral = Model();
esquinageneral.LoadModel("Models/esquinageneral.obj");
```

Para la segunda actividad, creo el modelo del holocrón moviendo las coordenadas en Y y Z y le asigné una articulación a cada pieza, y ya que la pieza original está horizontal, al rotarla conserva el eje Y en su posición original, así que al rotar en ese eje con el teclado girará sin afectar la trayectoria.

```
// HOLOCRON
//Modelo Inicial
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::mat4(1.0);
model = modelaux;
color = glm::vec3(0.0f, 1.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.5f, -16.0f));
modelaux2 = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
baseholocron.RenderModel();
// Esquina 1 Rojo
model = modelaux2;
model = glm::translate(model, glm::vec3(5.1f, 5.2f, 5.1f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::rotate(model, -43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion14()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

```
// Esquina 2 Rosa
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f, 5.2f, -5.1f));
color = glm::vec3(1.0f, 0.0f, 1.0f);
model = glm::rotate(model, 135.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 3 Amarilla
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f, -5.2f, 5.1f));
color = glm::vec3(1.0f, 1.0f, 0.0f);
model = glm::rotate(model, 43.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -234.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion16()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 4 Naranja
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f, -5.2f, -5.1f));
color = glm::vec3(1.0f, 0.5f, 0.0f);
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -234.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion17()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

```
// Esquina 5 Verde
model = modelaux2;
model = glm::translate(model, glm::vec3(-5.1f, 5.2f, 5.1f));
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::rotate(model, -133.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -54.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion18()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 6 Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(5.2f, 5.1f, -5.2f));
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::rotate(model, 45.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -55.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion19()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
// Esquina 7 Morado
model = modelaux2;
model = glm::translate(model, glm::vec3(5.2f, -5.1f, 5.2f));
color = glm::vec3(0.5f, 0.0f, 0.5f);
model = glm::rotate(model, -225.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -235.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion20()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

```
// Esquina 8 Cafe
model = modelaux2;
model = glm::translate(model, glm::vec3(5.1f, -5.2f, -5.1f));
color = glm::vec3(0.8f, 0.5f, 0.2f);
model = glm::rotate(model, -138.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -235.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion21()), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
esquinageneral.RenderModel();
```

Posteriormente se importa el satélite utilizando 4 modelos.

```
//Satelite
Model satelitebase;
Model panel;
Model parabolica;
Model centro;
```

```
//Satelite
satelitebase = Model();
satelitebase.LoadModel("Models/satelitebase.obj");
panel = Model();
panel.LoadModel("Models/panel.obj");
parabolica = Model();
parabolica.LoadModel("Models/parabolica.obj");
centro = Model();
centro.LoadModel("Models/centro.obj");
```

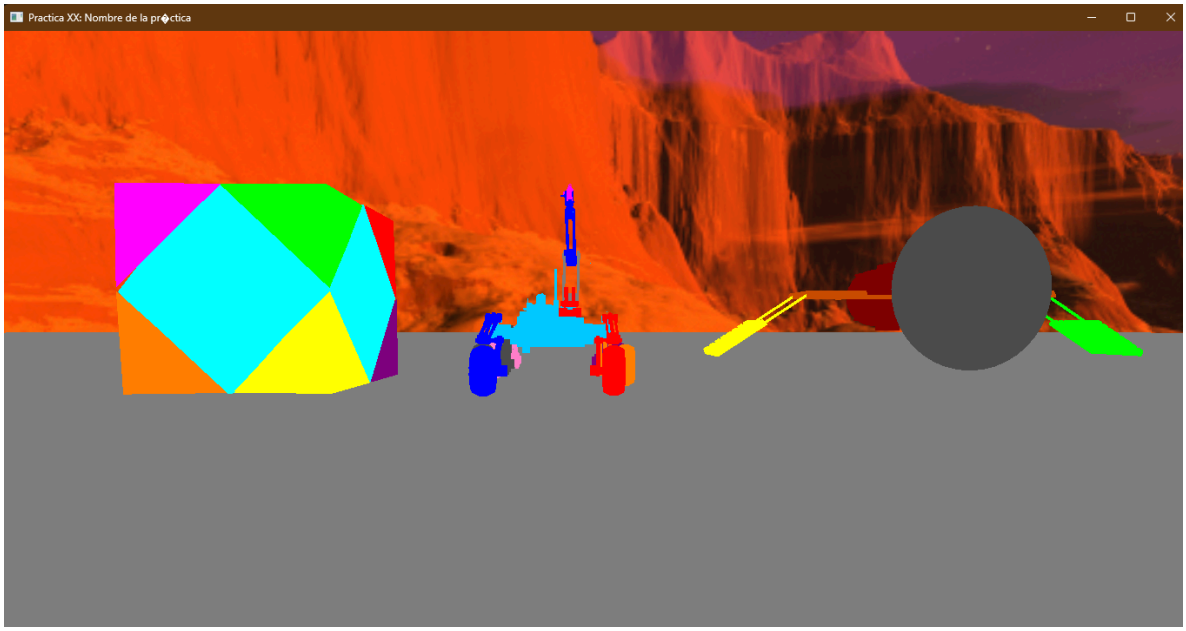
Creo la figura del satélite y agrego las rotaciones en X, Y y Z que se aplican a todo el modelo completo.

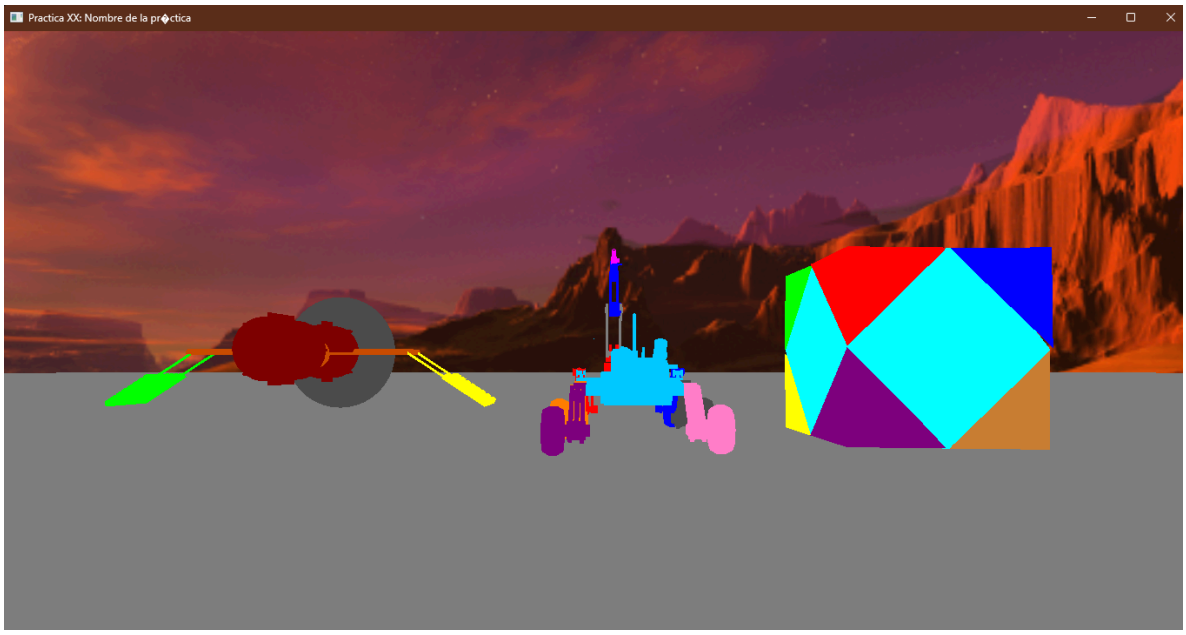
```
// SATELITE
// Base
color = glm::vec3(0.5f, 0.0f, 0.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.5f, 20.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
satelitebase.RenderModel();
// Antena Parabólica
model = modelaux;
model = glm::translate(model, glm::vec3(-7.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
color = glm::vec3(0.3f, 0.3f, 0.3f);
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
parabolica.RenderModel();
```

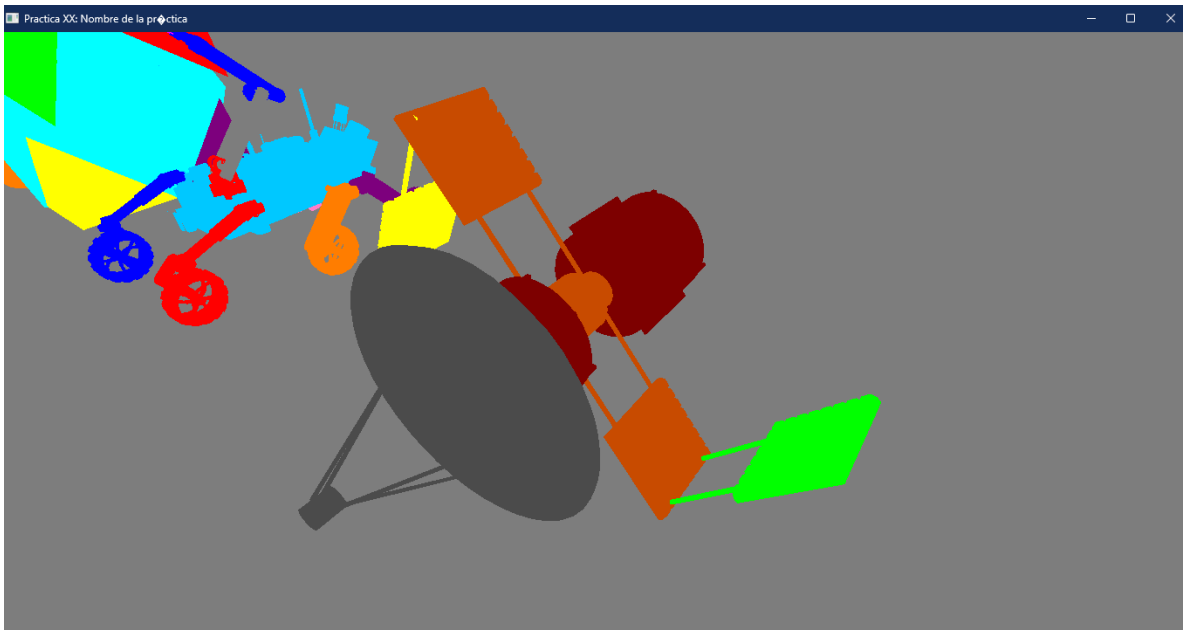
```
// Centro
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, 0.0f, 0.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(-1.0f, 0.0f, 0.0f));
modelaux2 = model;
model = modelaux2;
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
color = glm::vec3(0.8f, 0.3f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
centro.RenderModel();
// Panel Izquierdo
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.5f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel.RenderModel();
// Panel Derecho
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 6.5f));
model = glm::scale(model, glm::vec3(100.0f, 100.0f, 100.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion13()), glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
panel.RenderModel();
```

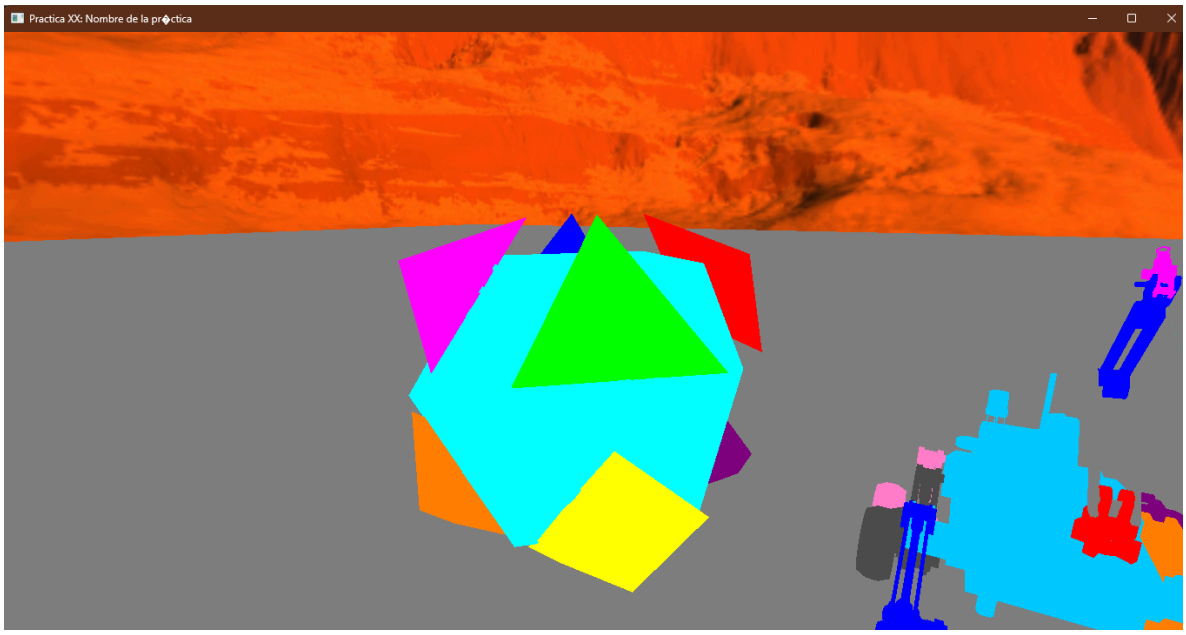
Agrego ejes en cada uno de los paneles, en el centro y en la antena parabólica.

La ejecución final se muestra a continuación:




Practica XX: Nombre de la pr◆ctica


Practica XX: Nombre de la pr◆ctica



2. Problemas presentados.

Solo me representó un problema el holocrón, fue muy tardado acomodar las esquinas.

3. Conclusión:

Gracias al desarrollo de esta práctica logré crear más figuras con modelado jerárquico y seguir utilizando las herramientas de Blender para separar los modelos en piezas y determinar los orígenes que fueran más convenientes para las rotaciones.